

MediPass Project Documentation

Project Overview

MediPass is a local facial-recognition appointment and digital token system. It is intended to help patients book an appointment, register their face, and later check in by camera or image upload to retrieve their digital token.

This document is for developers who have never worked on the project before. It explains the user flow, the technical flow behind each step, the current architecture, important implementation rules, demo data tools, production scaling considerations, and UAE/Abu Dhabi healthcare compliance concerns that must be reviewed before any real deployment.

The demo flow is:

1. A patient books an appointment.
2. The patient completes three guided face scans: straight, slightly left, and slightly right.
3. The backend extracts face embeddings from those scans using InsightFace.
4. The system stores the appointment, token, and embeddings.
5. The patient later checks in using a camera scan or uploaded image.
6. The backend compares the check-in face against patients that have exactly three usable face embeddings.
7. If the match is above the configured threshold, the token is returned.
8. Application-handled check-in attempts are logged.
9. The admin dashboard shows operational and recognition analytics.

The project currently uses:

- React + Vite frontend.
- Flask backend.
- InsightFace `buffalo_l` for face detection and embedding extraction.
- Local JSON files for storage.
- A demo seeder for selected VGGFace2-style test face folders.
- Unicorn, Nginx, and Docker Compose configurations for controlled demos.

Project Structure

```
project-root/
├── backend/
│   ├── face_service.py
│   ├── Dockerfile
│   ├── db/                # patients and check-in attempts
│   ├── demo_seed/seed_from_faces.py
│   ├── scripts/get_embedding.py
│   ├── threshold_tuning/tune_threshold.py
│   ├── tests/test_face_service.py
│   └── requirements.txt
├── frontend/
│   ├── Dockerfile
│   ├── nginx.conf
│   ├── src/               # app, booking, and dashboard
│   └── package.json
├── API.md
├── DEPLOYMENT.md
├── README.md
├── QUICK_START.md
└── docker-compose.yml
```

Key files:

- `backend/face_service.py` : Flask API for face analysis, registration, verification, and logging.
- `frontend/src/App.jsx` : Main navigation and check-in flow.
- `frontend/src/BookingFlow.jsx` : Appointment form and guided three-angle registration.
- `frontend/src/AdminDashboard.jsx` : Admin analytics and drill-down tables.
- `backend/demo_seed/seed_from_faces.py` : Creates demo patients and check-in logs from prepared face folders.
- `backend/tests/test_face_service.py` : API regression tests with a fake model and temporary JSON.
- `backend/db/` : Local patient, appointment, embedding, token, and check-in attempt storage.
- `docker-compose.yml` and `DEPLOYMENT.md` : Demo container and Linux service configuration.

The demo containers serve the frontend through Nginx and run the API with one Unicorn worker and four threads. A named volume preserves the InsightFace model cache. This configuration is for controlled demonstrations, not public production use.

Local Development

Use Python 3.10 or newer and Node.js 22.12 or newer; Node.js 24 is recommended.

Backend

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python face_service.py
```

The backend runs at `http://localhost:5050` .

On first run, InsightFace may download the `buffalo_l` model.

Frontend

In another terminal:

```
cd frontend
npm ci
npm run dev
```

The frontend runs at `http://localhost:5173` .

Dates

User-facing dates use:

```
dd/mm/yyyy, HH:mm
```

Appointment inputs and JSON storage use ISO-style strings for reliable parsing and sorting. Creation and check-in timestamps include a timezone offset; `appointment_time` may be stored without one because the booking form sends local appointment date and time.

For example, `15/07/2026, 09:00` is displayed while `2026-07-15T09:00:00` is stored or returned by the API.

System Flow

Step 1: Main Menu

The React app starts in `frontend/src/App.jsx` and shows three choices:

- Book Appointment
- Check In
- Admin Dashboard

The app uses a simple `mode` state:

```
menu | booking | checkin | admin
```

The frontend API URL is configured with:

```
import.meta.env.VITE_API_URL || "http://localhost:5050"
```

Local development uses `http://localhost:5050` ; production can set `VITE_API_URL` .

Step 2: Appointment Details

The patient enters name, doctor, department, appointment date, and appointment time in `frontend/src/BookingFlow.jsx` .

Both the frontend and backend reject invalid dates and appointment times that are not in the future.

The date field is a custom `dd/mm/yyyy` text field instead of browser-native `datetime-local` , so it consistently displays in UAE-friendly order. Before sending to the backend, the frontend combines the date field and time field:

```
15/07/2026 + 09:00
```

into:

```
2026-07-15T09:00
```

Step 3: Three-Angle Face Registration

The patient completes three guided scans:

1. Straight
2. Slightly left
3. Slightly right

The project requires exactly three usable face embeddings per registered patient:

```
REQUIRED_FACE_SAMPLE_COUNT = 3
```

The `/patients` endpoint reports a patient as registered only when `face_embedding_count == 3`. Registration, matching, seeding, and dashboard logic all assume exactly three samples, so records with any other count are treated as incomplete.

Frontend Capture Flow

The guided capture code is in `frontend/src/BookingFlow.jsx`.

For each angle, the frontend:

1. Opens the webcam with `navigator.mediaDevices.getUserMedia`.
2. Draws preview frames into a hidden canvas.
3. Converts frames to base64 JPEG.
4. Sends smaller preview frames to `/analyze-face-pose`.
5. Waits for several stable `ready` checks.
6. Captures the full-quality image and moves to the next angle.

Preview frames are smaller to reduce request size. The higher-quality final captures are sent after all three angles are complete for embedding extraction.

Backend Pose Analysis

Pose analysis is handled by:

```
POST /analyze-face-pose
```

For pose analysis, the backend decodes the preview image, runs InsightFace detection, and checks that:

- exactly one face is visible
- face yaw matches the requested angle
- pitch and roll are within limits
- face is centered
- face size, detection confidence, and image sharpness are high enough

The response includes `ready`, a message, pose values, quality values, and detailed check results.

If the scan is not ready, the backend returns instructions such as:

- Move your face to the center.
- Move closer to the camera.
- Keep your head level.
- Hold still for a sharper scan.
- Turn slightly left.
- Turn slightly right.

Angle Strictness

The yaw ranges are configured in `backend/face_service.py`:

```
POSE_YAW_SIGN = int(os.getenv("POSE_YAW_SIGN", "-1"))
POSE_TARGETS = {
    "forward": {"yaw": (-8, 8)},
    "left": {"yaw": (-30, -20)},
    "right": {"yaw": (20, 30)},
}
```

The target ranges apply after `POSE_YAW_SIGN` is applied. Set it to `-1` or `1` for the registration camera and flip it when real-camera testing shows left and right are reversed. Tests accept either setting rather than enforcing one camera orientation. A shared backend serving differently oriented cameras needs per-device calibration before production use.

Step 4: Booking And Registration Save

After all three images are captured, the frontend sends:

```
POST /book-with-face-set
```

The request includes:

- name
- doctor
- department
- appointment_time
- booking_request_id : stable client-generated idempotency key
- images : three ordered base64 images, maximum 6 MB decoded each

The full request limit is 20 MB. Booking and registration are combined so incomplete patients are not created; the old `/book` and single-image `/register` endpoints remain disabled. Final images repeat the forward, left, and right pose and quality checks server-side, so arbitrary images cannot bypass the guided capture.

The backend:

1. Validates required text, a future ISO appointment time, and three ordered images.
2. Rechecks pose and quality, requires one face, and extracts each embedding.
3. Checks that all three embeddings plausibly belong to the same person.
4. Rejects the entire request if any validation fails.
5. Generates IDs and a token, atomically saves JSON, and returns the result.

Each saved patient record contains the generated IDs, appointment details, digital token, `token_issued`, timestamps, `registered: true`, and `face_embeddings`.

The important stored shape is:

```
patient_id, appointment_id, digital_token
appointment_time, doctor, department
token_issued, created_at, registered_at
face_embeddings: exactly three 512-number vectors
booking_request_id: optional internal retry key
```

The save is all-or-nothing. If one image fails, the full booking is rejected. Repeating a successful request with the same `booking_request_id` returns the original booking instead of creating a duplicate. The internal retry key and raw embeddings are hidden from `/patients`.

Step 5: Check-In

The patient chooses **Check In** in `frontend/src/App.jsx`.

They can check in with a webcam capture or uploaded image.

The frontend sends the image to:

```
POST /verify
```

If successful, the UI shows:

- digital token, patient name, and patient ID
- appointment time, doctor, and department
- recognition similarity score, displayed as a percentage

If verification fails, the UI shows the failure message and does not reveal a token. The backend still logs the attempt so the dashboard can report failed-match patterns.

The backend response retains `match_label` and `match_type` for diagnostics, but the polished success screen no longer displays the internal match-angle details.

Step 6: Face Verification

The backend `/verify` endpoint:

1. Validates that an image exists.
2. Decodes the image.
3. Extracts a face embedding with InsightFace.
4. Loads patients from `patients.json`.
5. Builds comparison vectors for each patient with exactly three usable embeddings.
6. Calculates cosine similarity.
7. Chooses the best-scoring patient.
8. Checks the score against `SIMILARITY_THRESHOLD`.
9. Marks the token as issued if accepted.
10. Logs the attempt.
11. Returns the token or a failure response.

Match Vectors

Each valid patient stores three embeddings:

- forward
- left
- right

At check-in, the backend compares against seven vectors per patient:

1. forward
2. left
3. right
4. forward-left average
5. forward-right average
6. left-right average
7. all-angles average

The best of these seven scores becomes that patient's score. The highest patient score overall is then compared with the threshold.

The averaged vectors help when a check-in face is between two registered angles. Averages are normalized before comparison.

Similarity Threshold

Default:

```
0.45
```

Override:

```
SIMILARITY_THRESHOLD=0.47 python face_service.py
```

Higher threshold:

- reduces false accepts
- increases false rejects
- is safer for identity-sensitive use

Lower threshold:

- increases successful matches
- increases wrong-person risk
- is more convenient but less strict

Healthcare check-in should prioritize avoiding incorrect patient identification, so threshold tuning should be based on real local test data.

Registration also requires every pair in the three-image set to meet `REGISTRATION_SIMILARITY_THRESHOLD`, which defaults to `0.20`. Similarity values are model scores, not calibrated probabilities.

Step 7: Token Issuing

If the match passes the threshold, the backend:

1. Sets `token_issued` to `true`.
2. Writes `token_issued_at`.
3. Saves the patient database.
4. Returns the digital token.

The current token model is simple: the token is created during booking and revealed during successful check-in. In production, token issuing should integrate with the hospital queueing or appointment system.

Step 8: Check-In Attempt Logging

Each `/verify` attempt handled by the application appends an entry to:

```
backend/db/checkin_attempts.json
```

This includes both successful and failed attempts.

Common fields:

- attempt ID
- timestamp
- success
- reason
- confidence
- threshold
- patient ID, appointment ID, doctor, and department when available
- error message when applicable

Common reasons:

- matched
- below_threshold
- no_registered_embeddings
- no_face
- multiple_faces
- invalid_image
- missing_image
- invalid_request

Check-in attempts are stored separately from patients because they are analytics and audit-style events, not patient master data.

Step 9: Admin Dashboard

The admin dashboard is implemented in:

```
frontend/src/AdminDashboard.jsx
```

It fetches:

```
GET /patients
GET /checkin-attempts
```

The `/patients` endpoint removes raw face embeddings before returning patient records. Embeddings are sensitive biometric data and should not be sent to the UI unless strictly necessary.

Current dashboard cards:

- Total Patients
- Upcoming Appointments
- Today's Check-in Attempts
- Today's Successful Check-ins
- Average Successful Match
- Average Failed Match
- Tokens Issued
- Average Days Booked In Advance
- Check-in Success Rate
- Most Common Failure
- Low-Confidence Successes
- Near-Miss Failures

Most cards are clickable. Clicking a card opens a filtered table showing the records behind that metric.

Useful calculations:

- Check-in success rate: $\text{successful attempts} / \text{total attempts}$.
- Average successful match: average confidence for successful attempts.
- Average failed match: average confidence for failed match attempts with numeric confidence.
- Average days booked in advance: appointment date minus booking creation date.
- Low-confidence successes: successful matches within 5 percentage points above the threshold.
- Near-miss failures: failed matches within 5 percentage points below the threshold.

Additional panels show failure reasons, check-ins over time, department breakdown, risky matches, and repeated failures. Low-confidence successes and near-miss failures are especially useful for threshold tuning.

Demo Data And Seeder

The seeder lets developers test the system without manually booking many patients.

Script:

```
backend/demo_seed/seed_from_faces.py
```

Expected folder layout:

```
backend/demo_seed/faces/  
  Person_Name/  
    forward.jpg  
    left.jpg  
    right.jpg  
  remaining/
```

The seeder uses only:

- forward.jpg
- left.jpg
- right.jpg

It ignores remaining/ . Use images in remaining/ later for manual check-in upload tests.

By default, the seeder:

1. Reads prepared face folders.
2. Requires exactly three selected registration images per person.
3. Extracts real InsightFace embeddings.
4. Skips people whose images fail or whose three embeddings are inconsistent.
5. Removes previous records created by the seeder.
6. Keeps non-demo records.
7. Adds demo patients.
8. Adds dashboard-friendly check-in attempt logs by default.
9. Backs up existing JSON files before writing.

Useful commands:

```
cd backend  
python demo_seed/seed_from_faces.py --dry-run  
python demo_seed/seed_from_faces.py  
python demo_seed/seed_from_faces.py --clear-all  
python demo_seed/seed_from_faces.py --clear-seeded-only  
python demo_seed/seed_from_faces.py --clear-all-only
```

Command behavior:

- --dry-run : show detected people without writing.
- default: clear previous seeded data, keep real data, then seed.
- --clear-all : clear all patient and attempt data, then seed.
- --clear-seeded-only : clear only seeded records without seeding.
- --clear-all-only : clear all records without seeding.

Threshold Tuning

Threshold tuning exists separately from the live app so it can be removed or ignored later.

The goal is to estimate:

- how often the correct person is rejected
- how often the wrong person is accepted
- which threshold best fits the desired risk level

Important terms:

- False reject: the real patient is rejected.
- False accept: the wrong person is accepted.
- Hardest impostor: the most similar wrong person in the dataset.

For healthcare identity checks, minimizing false accepts is usually more important than maximizing convenience because incorrectly identifying one patient as another creates clinical, privacy, and operational risk.

Current Strengths And Limitations

Strengths

- Registration requires exactly three face samples.
- Booking and registration happen atomically.
- Final images repeat pose and quality validation server-side.
- Booking retries are idempotent when they reuse `booking_request_id`.
- Check-in attempts are logged separately.
- Admin dashboard uses real backend data.
- Dashboard cards support drill-down tables.
- Raw embeddings are hidden from `/patients`.
- Check-in supports camera and image upload.
- User-facing dates use `dd/mm/yyyy`.
- Similarity threshold is configurable.
- Demo seeding uses real embeddings, not fake vectors.

Limitations

- Storage uses local JSON files, not a database.
- Admin dashboard has no authentication or authorization.
- There is no patient consent flow.
- There is no formal admin audit trail.
- Face embeddings are stored directly in local JSON.
- Matching is linear search.
- There is no integration with hospital appointment, queueing, HIS, EMR, UAE Pass, Emirates ID, Malaffi, or Shafafiya systems.
- Encryption-at-rest, retention, deletion, monitoring, high availability, and incident response are not implemented.
- Liveness detection, rate limiting, and anti-spoofing controls are not implemented.

Scaling For Real Industry Use

Real-World Patient Flow

In a real Abu Dhabi clinic or hospital, MediPass should not be the source of truth for appointments. A realistic flow is:

1. Patient books through the hospital website, mobile app, call center, reception desk, or existing HIS/EMR.
2. The official scheduling system creates the appointment.
3. MediPass receives appointment data through an approved integration.
4. The patient is invited to register for face check-in after consent and identity verification.
5. Registration happens at a supervised kiosk, secure mobile app, or reception desk.
6. On appointment day, the patient checks in at a kiosk, reception tablet, mobile check-in station, or controlled entrance point.
7. MediPass verifies the face and retrieves the appointment/token context.
8. The queueing system issues or activates the token.
9. Staff handle exceptions manually.
10. Attempts are logged and admins review operational/security metrics.

Production Architecture

For production, split the system into separate services:

- frontend web or kiosk app
- backend API
- identity and access management
- appointment/HIS integration service
- face recognition service
- relational database
- vector index/search service
- audit logging service
- monitoring and alerting
- secure object storage only if images must be temporarily retained

Database And Matching

Replace JSON with production storage:

- PostgreSQL for patient, appointment, token, consent, and audit metadata.
- Vector index/database for embeddings.
- Redis for short-lived kiosk/session state.
- Object storage only if raw images are required; otherwise avoid storing raw face images after embeddings are created.

Likely tables:

- patients
- appointments
- face_registrations
- face_embeddings
- checkin_attempts
- tokens
- consent_records
- audit_events
- users
- roles
- facilities
- kiosks

Current matching is linear search:

```
check every patient, check seven vectors per patient
```

That is acceptable for demos and small pilots. For large deployments, use approximate nearest neighbor search such as FAISS, HNSW, pgvector, Milvus, Weaviate, or OpenSearch vector search.

At production check-in:

1. Extract embedding.
2. Query top vector candidates.
3. Re-rank with exact cosine similarity.
4. Apply threshold and business rules.
5. Accept, reject, or send to manual review.

Security, Privacy, And Operations

Production should include:

- TLS for data in transit.
- Encryption for databases and backups.
- Key management.
- No raw embeddings exposed to frontend clients.
- Role-based access control.
- Admin login and MFA for privileged users.
- Full audit logging.
- Rate limiting.
- Kiosk/device identity.
- Liveness or anti-spoofing checks.
- Manual fallback for failed recognition.
- Monitoring, alerting, backup, restore, and incident response.
- Data retention and deletion workflows.
- Stable kiosk lighting and camera placement.
- Support for multiple facilities and departments.

Privacy requirements should include:

- clear consent notice before registration
- explanation of what is collected and why
- retention period
- access and sharing explanation
- non-biometric alternative check-in method
- deletion request workflow
- consent version and timestamp
- access logs for viewing, exporting, updating, or deleting biometric data

UAE And Abu Dhabi Regulatory Notes

This section is not legal advice. Production use should be reviewed by UAE healthcare counsel, facility compliance teams, and relevant regulators.

Why Compliance Matters

The system processes:

- patient identity data
- appointment data
- healthcare-related operational data
- facial images during capture
- biometric face embeddings
- check-in logs

In the UAE, facial images and face embeddings can be biometric personal data because they can identify or confirm the identity of a person. Healthcare context can also make related records sensitive health data. Production use therefore needs privacy, cybersecurity, healthcare regulatory, and data residency review.

Key Sources

- UAE Personal Data Protection Law, Federal Decree-Law No. 45 of 2021: <https://uaelegislation.gov.ae/en/legislations/1972>
- UAE official data protection overview: <https://u.ae/en/about-the-uae/digital-uae/data/data-protection-laws>
- Federal Law No. 2 of 2019 on ICT in Health Fields: <https://uaelegislation.gov.ae/en/legislations/1209>
- Department of Health Abu Dhabi AAMEN and ADHICS V2: <https://www.doh.gov.ae/en/programs-initiatives/Aamen>
- Department of Health Abu Dhabi HIE Standards 2026: <https://www.doh.gov.ae/en/resources/HIE>
- Department of Health Abu Dhabi Shafafiya portal: <https://www.doh.gov.ae/en/Shafafiya>
- Malaffi policy page: <https://www.malaffi.ae/what-is-malaffi/policies/>

UAE Personal Data Protection Law

The UAE PDPL defines personal data broadly and includes sensitive personal data and biometric data. For this project, facial images and face embeddings should be treated as biometric personal data because they can identify or confirm the identity of a person.

Production implementation should include:

- privacy notice before face registration
- explicit biometric consent unless facility counsel approves another lawful basis
- consent records
- purpose limitation
- retention period
- access control
- correction/deletion/objection process
- complaint route
- processing records
- strong security controls
- cross-border transfer review

UAE Health ICT Law

Federal Law No. 2 of 2019 concerns ICT use in health fields. For this project, the key themes are confidentiality, electronic health system controls, health information exchange, UAE data hosting, and restrictions on storing or transferring health data outside the UAE.

Production implementation should:

- host health-related data inside the UAE unless transfer is legally approved
- avoid sending face embeddings or appointment data to overseas services without review
- use UAE-hosted infrastructure where possible
- follow DOH/MOHAP requirements for healthcare integrations
- treat check-in and appointment logs as health-related operational records

Abu Dhabi DOH, AAMEN, ADHICS, Malaffi, And Shafafiya

The Abu Dhabi Department of Health regulates healthcare facilities in Abu Dhabi. Its AAMEN programme and current ADHICS V2 materials address healthcare information security, patient privacy, resilience, governance, and cybersecurity maturity. Earlier standalone patient privacy and IoMT standards are marked as retired and addressed in ADHICS V2.

Production deployment should align with DOH/ADHICS expectations by implementing:

- security review before deployment
- asset and system inventory
- access control
- audit logging
- backup and disaster recovery
- incident response
- staff training
- privacy-by-design
- third-party processor review

Malaffi is Abu Dhabi's Health Information Exchange platform. The DoH publishes HIE Standards 2026 for current data exchange requirements, while Shafafiya contains exchange and transaction standards. MediPass should not connect directly without an approved use case, integration design, and compliance review. Normally it should integrate with the hospital scheduling/check-in system, which handles applicable HIE or Shafafiya obligations.

Biometric Controls For Production

Facial recognition should include:

- explicit biometric consent
- visible user-initiated check-in
- no silent background identification
- non-biometric alternative
- data minimization
- retention limits
- encryption
- audit logs
- deletion workflow
- model accuracy testing
- demographic performance testing
- human review for uncertain matches
- kiosk signage

Recommended Consent And Fallback Flow

Before registration, show:

1. What data is collected.
2. Why it is collected.
3. How it is used.
4. How long it is kept.
5. Whether raw images or embeddings are stored.
6. Whether data is shared.
7. How the patient can opt out or request deletion.
8. Who to contact for privacy questions.

Record:

- patient confirmation
- consent timestamp
- consent version
- operator or kiosk ID
- facility ID

Facial recognition should not be the only check-in method. Fallback options include Emirates ID, MRN or appointment number, phone verification, appointment QR code, or reception staff lookup.

Production Go-Live Checklist

Before real healthcare use:

- Replace JSON storage with production database and vector search.
- Add authentication, authorization, audit logs, and MFA where needed.
- Add consent capture, privacy notices, retention, and deletion workflows.
- Encrypt data at rest and in transit.
- Add liveness/anti-spoofing, manual fallback, and facility/kiosk identity.
- Add monitoring, backup/restore testing, and incident response.
- Validate threshold using local population data.
- Complete penetration testing, privacy risk assessment, hosting/data residency review, and DOH/UAE legal review.
- Complete security, recovery, operational, and clinical acceptance testing.

Developer Notes

Good next tasks:

- Add frontend tests for date parsing, camera cleanup, and booking flow errors.
- Add liveness, rate limiting, and stronger operational audit coverage.
- Add admin authentication, database abstraction, consent/retention configuration, and better threshold reports.

Current automated checks are the backend API regression suite plus frontend lint and production build. The backend tests do not load the real InsightFace model or alter demo data.

Be careful when changing `REQUIRED_FACE_SAMPLE_COUNT` , `patient_match_vectors` , `set_patient_embeddings` , `/book-with-face-set` , `/register-face-set` , `/verify` , dashboard registered-patient logic, or the seeder folder structure. These pieces are connected.

Mental Model

BookingFlow.jsx	appointment form, 3 face captures, /book-with-face-set
face_service.py	validates images, stores patients, verifies check-ins, logs attempts
App.jsx	navigation, check-in UI, /verify, token display
AdminDashboard.jsx	patients + attempts analytics and drill-down tables
seed_from_faces.py	demo patients and attempt logs from prepared face folders

Summary

MediPass is a functional prototype of a face-assisted healthcare check-in system. It guides users through three face-angle captures, stores exactly three embeddings, matches against real and derived face vectors, returns a token only above threshold, logs every check-in attempt, and shows admin analytics.

For Abu Dhabi/UAE healthcare deployment, it still needs production storage, authentication, consent management, privacy controls, audit logging, secure hosting, DOH/ADHICS alignment, UAE data protection review, and hospital system integration.